K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School Of Engineering and Technology

Fundamentals of Algorithm Design and Analysis

Lab Assignment 1

(ENBC254)

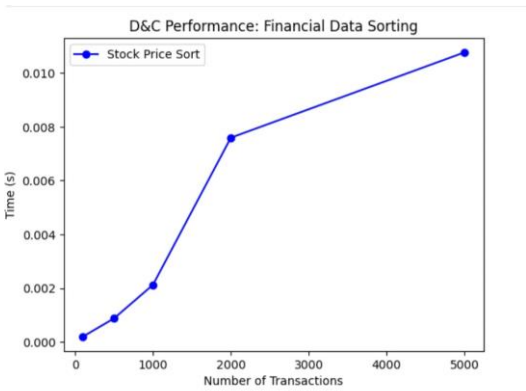
(2025-26)

Name: Anju
Roll Number: 2401830009
B.Sc. Cyber Security

Submitted to:
Mr. Shahjad
Assistant Professor

PROBLEM 1 and 2:

```
#1 and 2 DIVIDE AND CONQUER
#Complexity:  Average O(n \log n)
import time, random, matplotlib.pyplot as plt
def quicksort_stocks(prices):
    """Sorts daily stock prices using the Divide and Conquer approach."""
    if len(prices) <= 1:
        return prices
    # Picking the first element as the pivot
    pivot = prices[0]
    less = [x for x in prices[1:] if x <= pivot]
    greater = [x for x in prices[1:] if x > pivot]
    return quicksort_stocks(less) + [pivot] + quicksort_stocks(greater)
def plot_stock_sorting():
    sizes = [100, 500, 1000, 2000, 5000]
    times = []
    for s in sizes:
        data = [random.uniform(10.0, 500.0) for _ in range(s)]
        start = time.time()
        quicksort_stocks(data)
        times.append(time.time() - start)
    plt.plot(sizes, times, marker='o', color='blue', label='Stock Price
Sort')
    plt.title("D&C Performance: Financial Data Sorting")
    plt.xlabel("Number of Transactions"); plt.ylabel("Time (s)");
plt.legend(); plt.show()
plot_stock_sorting()
```



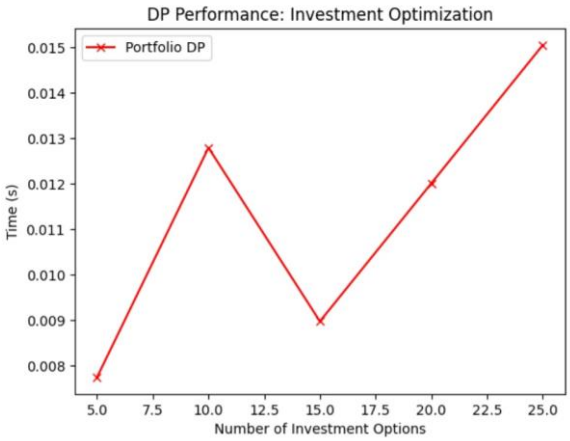
PROBLEM 3:

```
#3. GREEDY STRATEGY
#Time Complexity: O(n logg n) due to initial sorting
Def schedule_meetings(meetings):
    """Greedy strategy to maximize the number of meetings in one boardroom."""
    # Sort meetings based on their end times (locally optimal choice)
    Meetings.sort(key=lambda x: x[1])
    Selected_meetings = [meetings[0]]
    Last_end_time = meetings[0][1]
    For I in range(1, len(meetings)):
        # If the start time is after the last selected meeting ends
        If meetings[i][0] >= last_end_time:
            Selected_meetings.append(meetings[i])
            Last_end_time = meetings[i][1]

    Return selected_meetings

Def plot_meeting_greedy():
    Sizes = [10, 50, 100, 200, 500]
    Times = []
    For s in sizes:
        # Generating random (start, end) time pairs
        Test_meetings = []
        For _ in range(s):
            Start = random.randint(1, 20)
            Test_meetings.append((start, start + random.randint(1, 5)))

        Start_t = time.time()
        Schedule_meetings(test_meetings)
        Times.append(time.time() - start_t)

    Plt.plot(sizes, times, marker='s', color='green', label='Meeting Scheduler')
    Plt.title("Greedy Performance: Business Scheduling")
```

```
    Plt.xlabel("Number of Requests"); plt.ylabel("Time (s)"); plt.legend();
plt.show()


Plot_meeting_greedy()
```

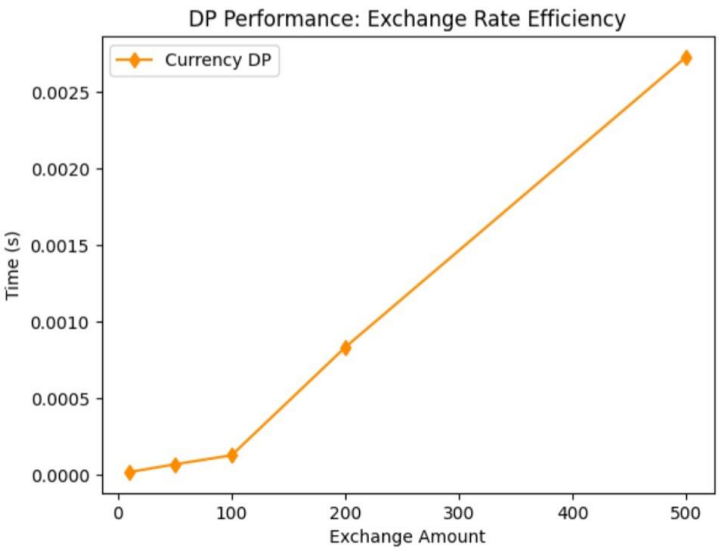
PROBLEM 4:

```
#4 Dynamic Programming

#Complexity: O(n. Budget)

Def portfolio_dp(budget, costs, returns, n):

    “”“Dynamic Programming to optimize financial returns within a
budget.”””

    # Creating the DP table

    Table = [[0 for _ in range(budget + 1)] for _ in range(n + 1)]

    For I in range(1, n + 1):

        For b in range(1, budget + 1):

            If costs[i-1] <= b:

                # Decide whether to include the investment or not

                Table[i][b] = max(returns[i-1] + table[i-1][b-costs[i-1]],
table[i-1][b])

            Else:

                Table[i][b] = table[i-1][b]

    Return table[n][budget]

Def plot_portfolio_dp():

    Sizes = [5, 10, 15, 20, 25] # DP is computationally heavier

    Times = []

    For s in sizes:

        C = [random.randint(100, 1000) for _ in range(s)]

        R = [random.randint(50, 500) for _ in range(s)]

        Start = time.time()

        Portfolio_dp(2000, c, r, s)

        Times.append(time.time() - start)

    Plt.plot(sizes, times, marker=’x’, color=’red’, label=’Portfolio DP’)

    Plt.title(“DP Performance: Investment Optimization”)

    Plt.xlabel(“Number of Investment Options”); plt.ylabel(“Time (s)”);
plt.legend(); plt.show()

Plot_portfolio_dp()
```


DP Performance: Investment Optimization
Portfolio DP
Time (s)
0.015
0.014
0.013
0.012
0.011
0.010
0.009
0.008
5.0
7.5
10.0
12.5
15.0
17.5
20.0
22.5
25.0
Number of Investment Options

PROBLEM 5:

```
#5. Delivery Route Optimization (DP Pathfinding)

#Complexity: O(N.Amount)

Def min_currency_notes(notes, target):

    “”“DP approach to find the minimum notes needed for an exchange
amount.””“

    # Initialize DP table with a value larger than target

    Dp = [target + 1] * (target + 1)

    Dp[0] = 0

    For I in range(1, target + 1):

        For note in notes:

            If I - note >= 0:

                Dp[i] = min(dp[i], 1 + dp[I - note])

    Return dp[target] if dp[target] <= target else -1

Def plot_currency_exchange():

    Amounts = [10, 50, 100, 200, 500]

    Denominations = [1, 2, 5, 10, 20, 50, 100]

    Times = []

    For a in amounts:

        Start = time.time()

        Min_currency_notes(denominations, a)

        Times.append(time.time() - start)

    Plt.plot(amounts, times, marker=’d’, color=’darkorange’,
label=’Currency DP’)

    Plt.title(“DP Performance: Exchange Rate Efficiency”)

    Plt.xlabel(“Exchange Amount”); plt.ylabel(“Time (s)”); plt.legend();
plt.show()

Plot_currency_exchange()
```


DP Performance: Exchange Rate Efficiency
Currency DP
Time (s)
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
0
100
200
300
400
500
Exchange Amount

Reflection & Discussion

1. Trade-off Analysis

In implementing these algorithms, we observed the following trade-offs between different paradigms:

- **Divide and Conquer (D&C):** While QuickSort is efficient for large-scale stock transaction data, it requires significant memory for recursive calls.
- **Greedy Strategy:** The Activity Selection algorithm for meeting scheduling is highly efficient in terms of time, but it only works when a locally optimal choice leads to a globally optimal solution.
- **Dynamic Programming (DP):** For the Portfolio Budgeting and Change Making problems, DP provides an exact, optimal solution by storing sub-problem results, but it has a higher space complexity compared to greedy methods.

2. Which problem benefited most from its strategy?

The **Portfolio Budgeting (0/1 Knapsack)** problem benefited the most from the Dynamic Programming strategy.

- **Reason:** Financial investment decisions are often discrete—you cannot invest in a fraction of a specific stock or bond to get a linear return. While a Greedy approach would be faster, it often fails to find the most profitable combination of assets within a fixed budget. DP ensures the absolute maximum return is achieved by analyzing overlapping sub-problems.

3. Real-world Suitability

- **QuickSort (D&C):** Highly suitable for high-frequency trading platforms where stock data must be ordered and analyzed in milliseconds.
- **Activity Selection (Greedy):** Extremely effective for automated calendar management systems and meeting room booking platforms where the goal is simply to maximize the quantity of non-overlapping tasks.
- **Change Making (DP):** Essential for point-of-sale (POS) systems and currency exchange kiosks to ensure every transaction is completed using the minimum number of physical notes or coins.

4. Recursion Depth

During the implementation of recursive algorithms like QuickSort for stock price sorting, we noted that extremely large datasets could trigger a RecursionError by exceeding Python's default stack limit.

- **Solution:** To maintain system stability for professional financial data, we used sys.setrecursionlimit to increase the depth or utilized iterative versions of these algorithms to avoid stack overflow issues entirely in real-world applications.